

Assignment 1: Mobile Robot Motion

Kevin Basta (501091628)

Gurkarn Dhaliwal (501108633)

Teodor Cirstoiu (501115733)

Syed Maisam Abbas (501103255)

CPS607: Autonomous Mobile Robotics

Section 1

Dr. Sven Lilge

February 13, 2026

Table of Contents

Part 1 – Wheeled Mobile Robot Design.....	3
Question 1.1.....	3
Question 1.2.....	3
Question 1.3.....	4
Part 2 - Robot Motion Model and Calibration.....	6
Question 2.1.....	6
Question 2.2.....	7
Part 3 - Open-Loop Path Tracking.....	11
Question 3.1.....	11
Question 3.2.....	14
Question 3.3.....	17
Appendix.....	18
References.....	20

Part 1 – Wheeled Mobile Robot Design

Question 1.1

Draw a top-down sketch of your mobile robot showing the wheel types and wheel configuration, using the notation and visualization conventions introduced in class. Clearly label all relevant components. For each wheel, specify its type and indicate whether it is steered, powered, or passive.

The figure below follows the format/diagrams shown in the lecture 3 slides (Lilge, “Wheeled Mobile Robots”).

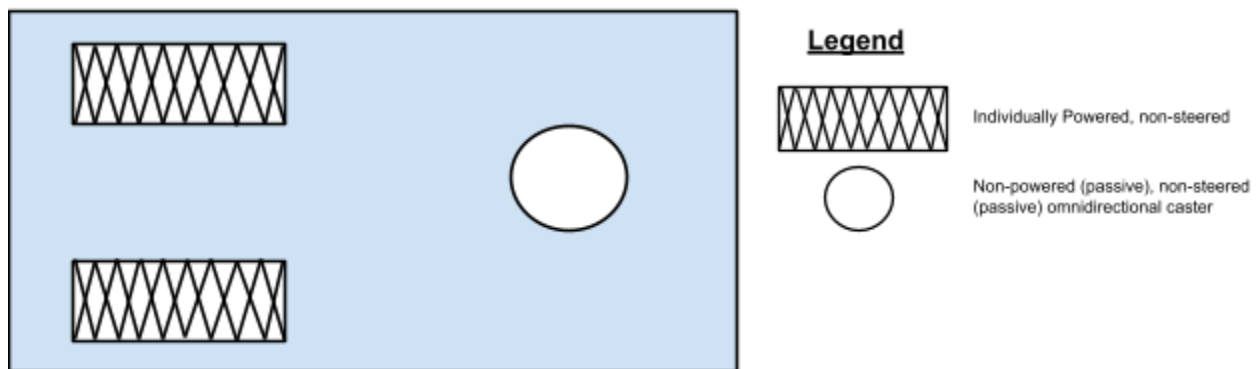


Figure 1. The top-down sketch of the mobile robot showing the wheel configuration. The two powered wheels are independently driven, and a passive caster wheel provides additional support.

Question 1.2

Discuss the stability of your robot. Explain whether the robot is stable or unstable, and justify your reasoning by describing the design features that contribute to its stability.

Our robot is stable, specifically, it's statically stable. This is because we have three wheels (contact points), and the center of gravity of the robot lies within the triangle formed by those contact points (Lilge, "Wheeled Mobile Robots"). The center of gravity of our robot lies within the triangle formed by our three wheels as most of our electronics and parts are placed within that triangle. If we put too much weight outside that triangle, our robot will become unstable (Lilge, "Wheeled Mobile Robots").

Question 1.3

Discuss the advantages and disadvantages of your robot design in terms of maneuverability and controllability. Identify one key advantage and one key disadvantage overall. Then, analyze the design trade-offs by describing ways to make the robot either more maneuverable or more controllable, and explain the trade-offs associated with each modification.

Our robot design, which is a differential drive robot with two separately powered wheels and one caster wheel, has good maneuverability and controllability characteristics.

One key advantage that spans both maneuverability and controllability is that our robot is able to change its direction quickly, unlike ackerman style robots. This is due to the fact that we can rotate one powered wheel clockwise while rotating the other wheel counterclockwise to effectively change the robot's direction. Our robot design strikes a good balance between maneuverability and controllability considering the fact that higher maneuverability results in lower controllability (Lilge, "Wheeled Mobile Robots").

One key disadvantage of our robot is that even though we have good controllability, the robot suffers from sensitivity to wheel/motor speed mismatches because the two wheels are powered separately and are not connected. We found this out as we were trying to get the robot to go straight for the purposes of the lab. This, along with the unpredictability of the caster wheel, makes it difficult to maintain straight lines in the real world without sensors, therefore impacting the actual controllability of the robot negatively.

To make the robot even more maneuverable, we can replace our three wheels with Swedish wheels. This would make our robot fully omnidirectional. The tradeoff for this modification is that the controllability of our robot will suffer, as the motion of our robot would require more inputs and more complex algorithms and coordination of motors to perform even relatively simple motions such as going straight (Lilge, “Wheeled Mobile Robots”).

To make our robot more controllable, we can connect our two powered wheels (like in Ackerman style), and swap out our caster wheel for a non-powered wheel that we can steer/change the direction of through control inputs. This would simplify our control of the robot, making the inputs required only forward velocity and steering angle like in Ackerman style robots. The tradeoff for this modification would be that our maneuverability will suffer, as we would no longer be able to change direction in place as we can with our current differential drive design (Lilge, “Wheeled Mobile Robots”).

Part 2 - Robot Motion Model and Calibration

Question 2.1

State the forward and inverse differential kinematics and motion model of your robot, given its physical dimensions (e.g., wheel radius and wheelbase). Assume the robot can be modelled as a differential-drive system. Briefly explain the purpose of each model, what they compute, and what purpose they serve in mobile robotics.

Our physical dimensions are:

- **Radius:** $r = 1.25 \text{ in} = 0.03175 \text{ m}$
- **Wheelbase:** $b = 2.625 \text{ in} = 0.066675 \text{ m}$

Differential kinematics describes the mapping between wheel velocities to the robot's body-centric velocities (Lilge, "Mobile Robot Kinematics"). The Inverse Differential Kinematics model returns (Q1, Q2), where Q1 is the rotational velocity of the right wheel, and Q2 is the rotational velocity of the left wheel (Lilge, "Mobile Robot Kinematics"). We get them by using the body-centric linear velocity and angular velocity that we input (Lilge, "Mobile Robot Kinematics"). This model is used to calculate the wheel speeds needed to make the robot move at the desired speed and turning rate.

The Inverse Differential Kinematics model with our physical dimensions r and b :

$$\begin{aligned} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} &= \begin{bmatrix} 1/r & b/r \\ 1/r & -b/r \end{bmatrix} \begin{bmatrix} V_x \\ \omega \end{bmatrix} \\ &= \begin{bmatrix} 1/0.03175 & 0.066675/0.03175 \\ 1/0.03175 & -0.066675/0.03175 \end{bmatrix} \begin{bmatrix} V_x \\ \omega \end{bmatrix} \end{aligned}$$

$$= \begin{bmatrix} 31.496 & 2.1 \\ 31.496 & -2.1 \end{bmatrix} \begin{bmatrix} V_x \\ \omega \end{bmatrix}$$

The Forward Differential Kinematics model returns (V_x, ω) where V_x is the body-centric linear velocity of the robot and ω is the body-centric angular velocity of the robot (Lilge, “Mobile Robot Kinematics”). We get them by using the rotational speed of the wheels (Lilge, “Mobile Robot Kinematics”). This model tells us how fast the robot is moving and turning based on the provided wheel speeds.

Forward Differential Kinematics model with our physical dimensions r and b :

$$\begin{aligned} \begin{bmatrix} V_x \\ \omega \end{bmatrix} &= \frac{1}{2} \begin{bmatrix} r & r \\ r/b & -r/b \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} \\ &= \frac{1}{2} \begin{bmatrix} 0.03175 & 0.03175 \\ 0.03175/0.066675 & -0.03175/0.066675 \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} \\ &= \frac{1}{2} \begin{bmatrix} 0.03175 & 0.03175 \\ 0.476 & -0.476 \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} \end{aligned}$$

Question 2.2

Propose and implement a method to determine an approximate mapping between PWM duty cycle and resulting wheel velocity, assuming a linear relationship. Then, use this mapping to modify your forward kinematics and motion model so that they accept PWM commands as inputs instead of wheel velocities.

Our proposed method to determine an approximate mapping between PWM duty cycle and the resulting wheel velocity is to set the PWM duty cycle to a given value, and measure how long the robot takes to cover 1 meter of distance. Since velocity can be expressed as a ratio between the distance and change in time, that is, $\frac{d}{\Delta t}$ (Lilge, “Mobile Robot Kinematics”), we can divide the distance travelled by the time it took our robot to cover that distance to get the velocity and map it to the PWM duty cycle we tested (Lilge, “Mobile Robot Kinematics”). Then using our Inverse Differential Kinematics model we will find the resulting wheel angular velocities. We can then collect data about the wheel angular velocity at different PWM duty cycle values and fit a linear line to our dataset (Lilge, “Mobile Robot Kinematics”). It might be helpful to fit a more complex curve to our dataset if the relationship is clearly not linear.

PWM / Volts	Distance	Time	Linear Velocity (Vx)	Angular Velocity (w)	$\begin{bmatrix} \phi_1 \\ \phi_2 \end{bmatrix} = \begin{bmatrix} 31.496 & 2.1 \\ 31.496 & -2.1 \end{bmatrix} \begin{bmatrix} V_x \\ \omega \end{bmatrix}$
255 PWM / 5.00 V	1 m	3.88 s	0.26 m/s	0 rad/s	Q = 8.189 rad/s
225 PWM / 4.41 V	1 m	4.26 s	0.23 m/s	0 rad/s	Q = 7.244 rad/s
200 PWM / 3.92 V	1 m	4.86 s	0.21 m/s	0 rad/s	Q = 6.614 rad/s
181 PWM / 3.55 V	1 m	5.63 s	0.18 m/s	0 rad/s	Q = 5.669 rad/s
170 PWM / 3.33 V	1 m	6.55 s	0.15 m/s	0 rad/s	Q = 4.724 rad/s
150 PWM / 2.94 V	1 m	8.29 s	0.12 m/s	0 rad/s	Q = 3.780 rad/s

Table 1. Body-Centric Measurements of Robotic Kinematics

Note: since the robot is moving straight, Q1 and Q2 (the angular velocity of the wheels), will be the same. The PWM signal is capped at 255 duty cycles.

Leveraging Desmos, we used Linear Regression to find the line of best fit for our six points. The formula for the line comes out as $y = 0.0419907(x) - 2.2355$ or $Q = 0.0419907(PWM) - 2.2355$. It should be noted that the speed of our robot changes with time due to the battery deteriorating, so these tests might not resemble the most up-to-date PWM mapping to wheel velocities.

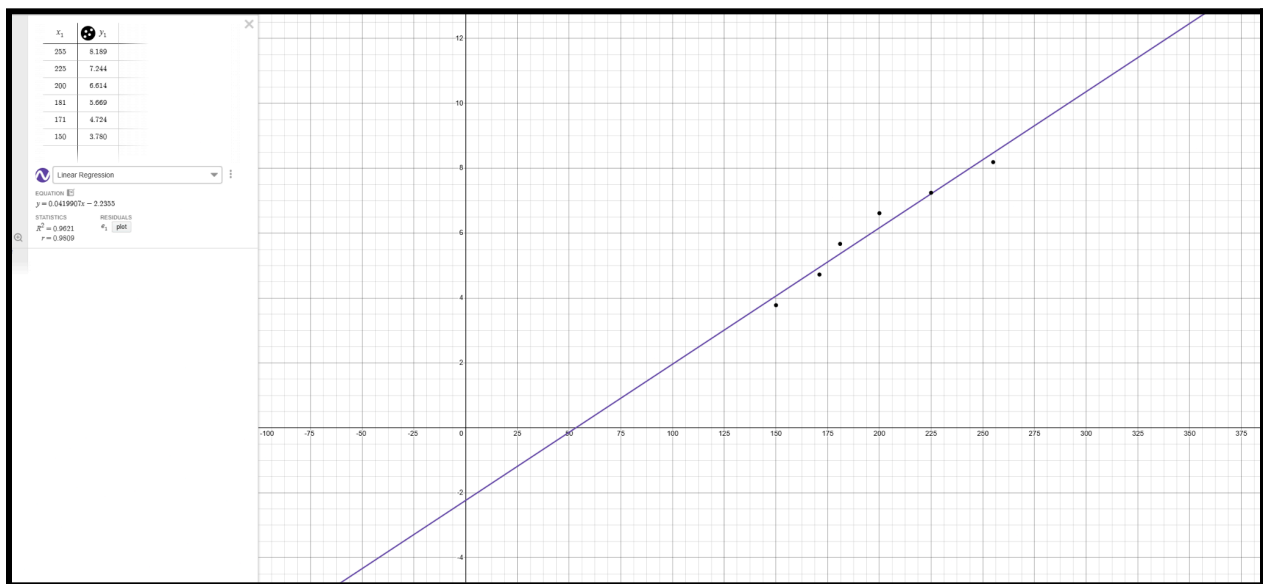


Figure 2. The relationship between PWM duty cycle and wheel angular velocity. A linear regression model is used to approximate the mapping.

Therefore, in our motion model and forward kinematic formula, we can replace Q1 and Q2 with a formula that depends on PWM1 and PWM2 (PWM per wheel). Our forward kinematic model will now look like:

$$\begin{bmatrix} V_x \\ \omega \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 0.03175 & 0.03175 \\ 0.03175/0.066675 & -0.03175/0.066675 \end{bmatrix} \begin{bmatrix} \phi_1 \\ \phi_2 \end{bmatrix}$$

$$\begin{bmatrix} V_x \\ \omega \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 0.03175 & 0.03175 \\ 0.03175/0.066675 & -0.03175/0.066675 \end{bmatrix} \begin{bmatrix} 0.0419907(PWM1) - 2.2355 \\ 0.0419907(PWM2) - 2.2355 \end{bmatrix}$$

And our motion model will now look like:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \mathbf{0} \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} + \underbrace{\begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} r/2 & r/2 \\ r/2b & -r/2b \end{bmatrix}}_{\mathbf{B(x)}} \underbrace{\begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \end{bmatrix}}_{\mathbf{u}}$$

$\dot{\mathbf{x}}$ $\mathbf{A(x)}$ $\mathbf{x}$ $\mathbf{B(x)}$ $\mathbf{u}$

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \mathbf{0} \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} + \begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0.015875 & 0.015875 \\ 0.238095 & -0.238095 \end{bmatrix} \begin{bmatrix} 0.0419907(PWM1) - 2.2355 \\ 0.0419907(PWM2) - 2.2355 \end{bmatrix}$$

Figure 3. The robot motion model incorporating the PWM-to-wheel angular velocity mapping within the forward kinematic equations.

Part 3 - Open-Loop Path Tracking

Question 3.1

Based on the differential kinematics model from Question 2 and the concepts covered in class, implement an open-loop (i.e., sensor-free) motion sequence that enables your robot to follow the given path as accurately as possible. Begin with the physical model previously derived, but you may refine or recalibrate the model to improve execution accuracy. After completing the path once, the robot should automatically begin another iteration (i.e., run in a continuous loop). Briefly describe your overall approach. What steps are required to convert the reference path into executable code and corresponding PWM motor commands? If applicable, explain any modifications or calibrations made to the kinematic model. As part of your submission, include a video demonstrating your robot executing the trajectory. The video must show at least three attempted laps of the robot following the route. Additionally, submit the full source code used to generate and execute the motion.

To implement the open-loop path tracking, we converted the reference path into a sequence of straight-line motions and 90-degree right turns. Since our robot does not have wheel encoders or any directional sensors, all the robot's motion was executed using PWM commands and timed delays (Lilge, "Sensors in Mobile Robotics"). The reference path was a 1 m x 1 m square. To follow this path, we programmed the robot to:

1. Move forward for a calculated duration of approximately 1m
2. Turn right approximately 90 degrees
3. Repeat this sequence

The robot repeats these steps infinitely many times, and every four loops (repetitions of the three steps above), it will have completed a lap around the reference path.

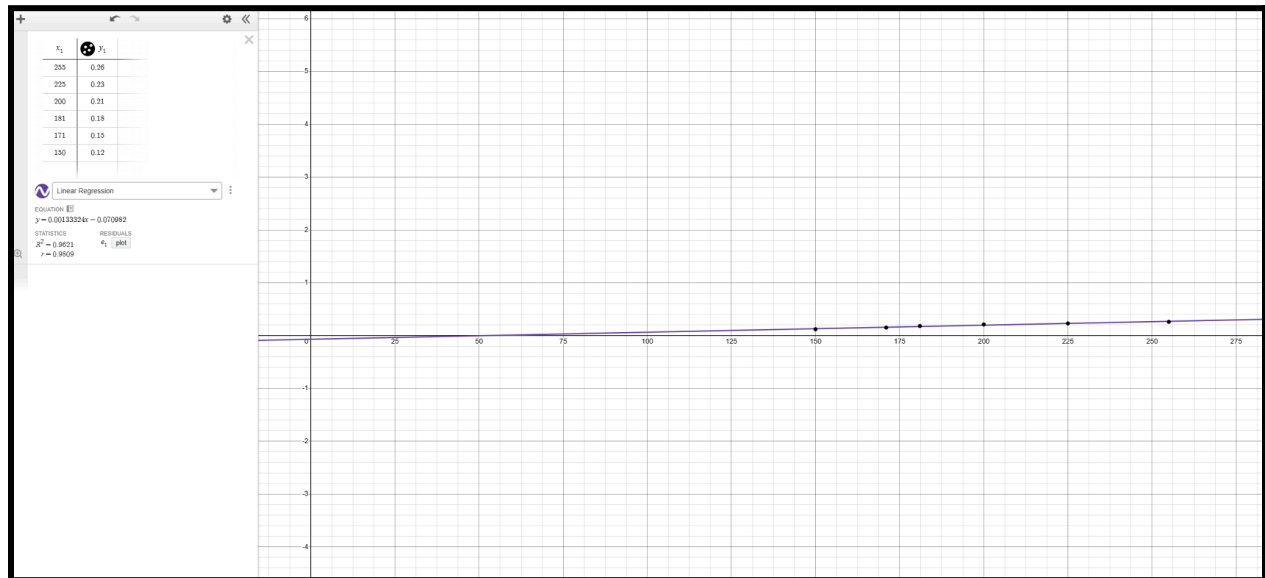


Figure 4. Linear relationship between PWM duty cycle and robot linear velocity.

As shown in the graph above (Figure 4), we derived a direct relationship between PWM input and the robot's measured linear velocity using data we gathered in question 2.2 in order to simplify our open-loop timing calculations.

$$V_x = 0.00133324(\text{PWM}) - 0.070982$$

We chose to use a forward PWM value of 225, and the theoretical linear velocity was:

$$V_x = 0.00133324(\text{PWM}) - 0.070982$$

$$V_x = 0.299979 - 0.070982$$

$$V_x = 0.228997 \approx 0.229 \text{ m/s}$$

We then determined the theoretical time to travel 1 meter:

$$T = d / V_x$$

$$T = 1/0.229$$

$$T = 4.3668 \approx 4.37 \text{ s}$$

Therefore, the theoretical time to travel 1 meter at PWM 225 was approximately 4.37 seconds (4370 ms). We used this value as our initial estimate for the forward motion duration in the code.

Although we began our practical implementation using our theoretical calculations, the actual timing was found through significant experimental adjustments. During testing, one battery completely lost charge and had to be replaced. The replacement battery we used was also not at full power. Since PWM motor controls are dependent on the voltage of their power supply, the robot's actual linear velocity differed from the predicted value. As battery voltage decreased, the robot moved slower and resulted in the theoretical timing not producing the expected 1 meter travel. As a result, we adjusted the straight movement duration experimentally until the robot was able to produce 1 meter travel with the battery we had available.

For turning, we did not perform a full theoretical calculation. Instead, we implemented an in-place right turn by moving the left wheel forward and the right wheel backward. The turn duration (TURN_RIGHT_MS) was tuned experimentally and was adjusted repeatedly until the robot achieved an approximate 90-degree rotation.

We also observed some drift during the straight motion due to a mismatch between the two motor speeds. To mitigate this, we used scaling offsets (`leftOffset` and `rightOffset`) to modify the PWM applied to each motor. These offset values were tuned experimentally and stored in EEPROM to improve the robot's straight-line motion (SunFounder).

The final implementation of our code consists of timed straight motion segments, timed 90-degree turns, and brief pauses and small alignment motions to reduce errors from momentum as well as a misaligned caster wheel after in-place turns. The robot executes this sequence of motions continuously in the main loop of the program and completes multiple laps of the reference path.

Question 3.2

Qualitatively assess how well the robot tracked the desired path. Describe the challenges encountered during implementation, how you addressed them, and the likely sources of error or limitations that affected accuracy. Suggest how these limitations could be mitigated in future if both hardware and software could be modified.

Overall, the robot was able to follow the general $1\text{ m} \times 1\text{ m}$ square path and complete multiple laps. The square shape was recognizable, but accuracy decreased over time. The first lap was relatively accurate. After the second lap, the errors became much more noticeable. Small position and heading errors accumulated at each straight segment and turn, causing the robot to drift progressively farther from the intended path.

One of the main challenges was significant variation between runs. In some trials, the robot drifted slightly to the right during straight motion. We attempted to correct this by adjusting the motor offsets and slightly reducing the PWM applied to one wheel. In later trials, the same adjustment sometimes caused drift in the opposite direction. This made it difficult to determine whether the error was caused by motor mismatch or by other factors such as wheel slip, surface friction, or caster wheel behaviour. Because the robot operates fully open-loop and does not use feedback, we were unable to isolate the exact source of the drift.

Battery voltage also had a major impact on performance. One battery completely lost charge during testing and had to be replaced. The replacement battery was not fully charged, which introduced further inconsistencies. Since PWM motor control depends directly on supply voltage, changes in battery level affect both the robot's linear speed and turning rate. As the battery weakened, the robot moved more slowly, and the previously calculated timing values no longer produced 1 meter of travel. This required repeated adjustments to both the straight and turning durations.

Turning was especially sensitive to these effects. Changes in battery voltage or floor friction caused the robot to under-rotate or over-rotate during the 90-degree turns. In addition, the caster wheel sometimes remained slightly misaligned after an in-place turn. If the caster was not properly aligned before the next straight segment, it introduced a small heading error at the start of motion. To reduce this effect, we added a short forward "alignment" movement before the longer straight segment. This allowed the caster wheel to settle into the correct orientation

before continuing forward. While this reduced some heading error, it did not eliminate accumulated drift over multiple laps.

The primary limitations affecting the robot's accuracy were:

- Absence of wheel encoders for velocity feedback, which we know would help per (Lilge, "Sensors in Mobile Robotics")
- No measurement of heading or orientation, which we know would help per (Lilge, "Sensors in Mobile Robotics")
- PWM output dependent on battery voltage
- Motor speed mismatch between wheels
- Wheel slip and surface friction variability
- Caster wheel misalignment during in-place turns

If hardware and software modifications were permitted, accuracy could be improved by incorporating wheel encoders and implementing closed-loop velocity control for each motor (Lilge, "Sensors in Mobile Robotics"). Adding a gyroscope or IMU would allow heading correction during both straight motion and turning (Lilge, "Sensors in Mobile Robotics"). Additionally, using a more stable power source or monitoring battery voltage to compensate for supply variations would improve consistency and repeatability.

Question 3.3

Demonstrate your implemented approach to the TA during lab hours, again showing at least three laps of the path. The grade will be composed as follows: 10 points for successfully running the robot and executing the task (within the expected limitations), and 5 points for demonstrating understanding through a brief explanation of the system to the TA.

The robot demo was presented on February 4th, 2026.

Appendix

Arduino Source Code for Open-Loop Square Trajectory Implementation

```
#include <EEPROM.h>

const int A_1B = 5;
const int A_1A = 6;
const int B_1B = 9;
const int B_1A = 10;

float leftOffset = 1.0;
float rightOffset = 1.0;
bool firstForward = true;

const int MOVE_FORWARD_MS = 3100;
const int ALIGNMENT_MOVE_FORWARD_MS = 100;
const int TURN_RIGHT_MS = 355;

void moveForward(int speed) {
    analogWrite(A_1B, 0);
    analogWrite(A_1A, int(speed * rightOffset));
    analogWrite(B_1B, int(speed * leftOffset));
    analogWrite(B_1A, 0);
}

void turnRight(int speed) {
    analogWrite(A_1B, int(speed * rightOffset));
    analogWrite(A_1A, 0);
    analogWrite(B_1B, int(speed * leftOffset));
    analogWrite(B_1A, 0);
}

void stopMovement() {
    analogWrite(A_1B, 0);
    analogWrite(A_1A, 0);
    analogWrite(B_1B, 0);
    analogWrite(B_1A, 0);
}

void square() {
    moveForward(225);
    if (!firstForward) {
```

```

    delay(MOVE_FORWARD_MS - ALIGNMENT_MOVE_FORWARD_MS);
} else {
    delay(MOVE_FORWARD_MS);
    firstForward = false;
}
stopMovement();
delay(600);
turnRight(220);
delay(TURN_RIGHT_MS);
stopMovement();
delay(600);
moveForward(225);
delay(ALIGNMENT_MOVE_FORWARD_MS);
stopMovement();
delay(400);
}

void setup() {
    Serial.begin(9600);
    pinMode(A_1B, OUTPUT);
    pinMode(A_1A, OUTPUT);
    pinMode(B_1B, OUTPUT);
    pinMode(B_1A, OUTPUT);
    EEPROM.write(0, 100); //write the offset to the left motor
    EEPROM.write(1, 97); //write the offset to the right motor
    leftOffset = EEPROM.read(0) * 0.01; //read the offset
    rightOffset = EEPROM.read(1) * 0.01; //read the offset
}

void loop() {
    square();
}

```

Note. The EEPROM-based motor speed offset calibration method was adapted from SunFounder, “Speed Calibration.” (SunFounder).

References

- Lilge, Dr. Sven. “Wheeled Mobile Robots.” *CPS607: Autonomous Mobile Robots*, Toronto Metropolitan University, 20 January 2026,
<https://courses.torontomu.ca/d2l/le/content/1113981/viewContent/6898781/View>.
Accessed 9 February 2026.
- Lilge, Dr. Sven. “Mobile Robot Kinematics.” *CPS607: Autonomous Mobile Robots*, Toronto Metropolitan University, 1 February 2026,
<https://courses.torontomu.ca/d2l/le/content/1113981/viewContent/6908006/View>.
Accessed 9 February 2026.
- Lilge, Dr. Sven. “Sensors in Mobile Robotics.” *CPS607: Autonomous Mobile Robots*, Toronto Metropolitan University, 9 February 2026,
<https://courses.torontomu.ca/d2l/le/content/1113981/viewContent/6914188/View>.
Accessed 9 February 2026.
- SunFounder. “11. Speed Calibration.” *SunFounder 3in1 Kit R4 Documentation*,
docs.sunfounder.com,
https://docs.sunfounder.com/projects/3in1-kit-r4/en/latest/car_project/car_speed_calibration.html. Accessed 9 February 2026.